Meta Learning-assisted Constraint Relaxation for Constrained Black-Box Optimization

Sijie Ma, Qianhao Zhu, Zhiyang Huang, Zeyuan Ma,
Hongshu Guo, and Yue-Jiao Gong(✉)

South China University of Technology
{masijie9, zhuqianhao324, scut.hzy, scut.crazynicolas,
guohongshu369, gongyuejiao}@gmail.com

Abstract. Constraint handling plays a key role in solving complex black-box optimization (BBO) problems with intricate constraints. Though intensively discussed, existing methods such as ϵ-relaxation rely heavily on human-defined hyper-parameters to ensure the optimization performance. To address this, in this paper, we propose meta learning dynamic, adaptive and generalizable ϵ-relaxation policy. Specifically, a bi-level learn-to-optimize framework is established. At the lower level, we modify a L-SHADE optimizer with controllable ϵ-relaxation policy, which combined with target constrained BBO problems, serves as an optimization environment. At the meta level, we propose a reinforcement learning-assisted control policy, which perceives the per-step optimization state of the low-level constrained optimization environment and suggests desired dynamic configuration (i.e., actions) for the low-level ϵ-relaxation policy accordingly. The meta-level policy is trained to control the low-level optimizer over a problem distribution, by maximizing the expected performance gain across the distribution. This leads to a generalizable optimizer that addresses constrained BBO problems via robust and flexible ϵ-relaxation policy. We validate its generalization performance on both synthetic and realistic testbeds. Further in-depth analysis provides interpretation on the proposed learning system. We provide sourcecodes of our project at https://github.com/MetaEvo/MeCO.

Keywords: Black-Box Optimization · Constraint Handling · Dynamic Algorithm Configuration · Reinforcement Learning.

1 Introduction

Constrained Black-Box Optimization (BBO) problems are characterized by the composition of target objective function and multiple constraint functions. Such problems are prevalent in real-world domains, e.g., automotive industry design [1] and physical/chemical processing [2]. A primary challenge of constrained BBO is that: the intricate constrains may cause the optimizer to keep away the focus on optimization to just seeking a feasible (i.e., valid) solution [3, 4]. Consequently, how to handle constraints to balance feasible solution finding and optimal solution searching draw much attention recently [5–7].

Various constraint handling techniques [8–10] are proposed to assist traditional optimization algorithms [11–13] with constraint optimization capability. Initially, penalty function method [14–16] are widely adopted due to the simplicity. Subsequently, multi-objective transformation methods [17–19] and surrogate-based methods [20–22] explicitly address the interplay between the objective and constraints, significantly improves the overall optimization performance. However, they also introduce additional algorithm complexity which requires deep expertise for the human-crafted designs. Recent ϵ-relaxation methods [23–26] strike a good balance between the algorithm complexity and constraint handling effectiveness through relaxation on the constraint violation. Therefore, the research objective of this paper focuses on ϵ-relaxation-based constrained BBO algorithms.

However, even for ϵ-relaxation methods, certain expertise is still needed to decide the concrete ϵ-relaxation value. In [23], the authors did preliminary tests on the target optimization problems and decided a fixed value. More recently, Fan et al. [24] pointed out that the ϵ-relaxation should be adaptively adjusted during the optimization to facilitate flexible exploration-exploitation tradeoff between the objective and constraints. They hence proposed an adaptive rule for that purpose. Nevertheless, such human-crafted design philosophy may fall shorts in true flexibility and generalization on unseen, noisy, complex constrained BBO problems. This naturally outlines an interesting research question: *Can we automate the ϵ-relaxation design to further boost optimization performance?*

Based on such research motivation, in this paper, we propose a **Me**ta learning-assisted optimization algorithm for **C**onstrained Black-Box **O**ptimization, termed as **MeCO**, which is capable of automatically learning flexible and generalizable ϵ-relaxation policy to further enhance solving efficacy on constrained BBO problems. Specifically, MeCO is modeled as a bi-level learn-to-optimize framework. At the lower level, a powerful L-SHADE [27] optimizer is modified with a controllable ϵ-relaxation policy for optimizing constrained BBO problems. At the meta level, MeCO employs a reinforcement learning (RL) [28] control agent as the ϵ-relaxation policy learner. At each step of the low-level L-SHADE optimization, the meta-level agent first perceives the optimization state of the low-level optimization environment, and then adjusts the ϵ-relaxation values in the low-level L-SHADE optimizer for each constraints to achieve dynamic tradeoff between objective space optimization or constraint space optimization. By regarding such dynamic configuration problem as Markov Decision Process (MDP), the meta-level RL agent is trained on a constrained BBO problem distribution, with the learning objective as maximizing the expected optimization performance across this distribution. Experimental results demonstrate that a well-trained MeCO optimizer achieves superior optimization performance to a broad range of representative human-crafted constrained BBO algorithms. It also presents surprisingly robust zero-shot generalization toward extremely challenging realistic scenarios (i.e., city-level UAV path planning tasks). Further in-depth analysis reveals interesting interpretation on what MeCO has learned and the importance of its design components.

The rest of this paper is organized as follows. In Sec. 2, we first introduce the conceptual aspects of constrained BBO and various solution paradigms and then review recent meta learning-assisted algorithm design works. In Sec. 3, we provide full details of MeCO, including its MDP modeling, meta-level RL agent, low-level optimization environment and training/inference workflow. In Sec. 4, we pose several key research questions and then present detailed empirical validation experiments and corresponding results analysis to demonstrate MeCO's effectiveness, generalization and design correctness.

2 Preliminary & Related Works

2.1 Constrained Black-Box Optimization

Without loss of generality, a Constrained Black-Box Optimization (BBO) Problem can be formulated as a minimization problem [29, 30]:

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}), \mathbf{x}^L \leq \mathbf{x} \leq \mathbf{x}^U, \\ \text{subject to} \quad & g_i(\mathbf{x}) \leq 0, \quad i = 1, \ldots, p, \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \ldots, q, \end{aligned} \tag{1}$$

where $\mathbf{x} \in \mathbb{R}^D$ is the decision vector of dimension D, $\mathbf{x}^L$ and $\mathbf{x}^U$ denote the lower and upper bounds of the search space, $g_i(\cdot)$ and $h_j(\cdot)$ denote p inequality constraints and q quality constraints respectively. An important metric widely used in existing constrained BBO algorithms is the violation value $\nu(x)$, which measure how good is a solution x aligns with the constraints. $\nu(x)$ is computed as the summation of all per-constraint violation:

$$\nu(x) = \sum_{i=1}^{p} max\{g_i(x), 0\} + \sum_{j=1}^{q} max\{|h_j(x)|, 0\} \tag{2}$$

Compared to unconstrained problems, Constrained BBO originates from a wide range of realistic scenarios and hence is more challenging due to the interplay between the objective function and complex constraints. To address constrained BBO problems, various constraint handling techniques are proposed and discussed in the literature [8–10]. Considering the specific constraint handling paradigms, these algorithms could be generally categorized into four groups. The first paradigm, as the most straightforward one, is penalty function method [14–16], which transforms a constrained optimization problem into an unconstrained one by adding the constraint violation value to the objective function: $f'(x) = f(x) + \lambda\nu(x)$, where penalty coefficient λ controls strength of contraint violation value. Then one can apply off-the-shelf optimizers for solving $f'(x)$. A potential limitation of this paradigm is that it may misleads the optimizers due to the mixed vague objective-constraint space. To avoid this, subsequent paradigms include multi-objective transformation methods [17] and surrogate-based methods [20–22]. The former regards the constraint functions of a problem

as additional objective, reshaping the constrained BBO as a multi-objective optimization problem: $\{f(x), g_1(x), ..., g_p(x), |h_1(x)|, ..., |h_q(x)|\}$. Then it leverages decomposition-based multi-objective scheme [18, 19] to support diversity management and explicitly resolve objective-constraint conflict during solving constrained BBO problems. The latter, in contrast, approximates black-box objective and constraints by differentiable surrogate models and then uses alternative gradient descent scheme to balance objective-constraint conflict. A representative surrogate-based method is COBRA [20]. Though promising, these two paradigms show more algorithmic complexity. Multi-objective transformation methods introduce additional cross-objective component and surrogate-based methods require extra expert-level knowledge on surrogate modeling field. Neither of these two paradigms are easy to design for human developers.

To maintain clear objective-constraint modeling and trivial algorithmic complexity, recent works proposed ϵ-relaxation methods [23–26], which utilizes a relaxation value ϵ to treat slightly infeasible solutions as feasible, achieving a flexible balance between objective optimization and constraint satisfaction. Specifically, given a re-defined (or adaptive) threshold ϵ, the violation value in a constrained BBO prolem is re-computed as:

$$\nu(x) = \sum_{i=1}^{p} max\{g_i(x) - \epsilon, 0\} + \sum_{j=1}^{q} max\{|h_j(x)| - \epsilon, 0\} \tag{3}$$

Compare to the original computation in Eq. 2, such relaxation allows the optimization process re-balance the importance of optimal searching and constraint minimization. More importantly, such paradigm shows much lower complexity and can be integrated into optimizers in a plug-and-play fashion.

2.2 Meta-Black-Box Optimization

Meta-Black-Box Optimization (MetaBBO) aims to improve the optimization performance of the low-level optimizer by meta-learning [31] an effective policy across a problem distribution [32–34]. While existing studies have explored supervised learning [35, 36], Neuroevolution [37, 38], and even LLMs [39–41] to learn such policy, the majority of MetaBBO approaches utilize RL/DRL [42] for key algorithm design tasks such as dynamic operator selection [43], algorithm selection [44] and algorithm configuration [45]. In such MetaBBO-RL paradigm, the algorithm design process within the low-level optimization process is modeled as Markov Decision Process (MDP). Given some specifically designed low-level optimization state at an optimziation step, the meta-level RL agent outputs desired algorithm design such as operator selection [46–49] or parameter control [50–54], or update rule generation [33, 55, 56]. For more concise introduction and review on diverse MetaBBO-RL methods, refer to the survey papers [32, 42, 57]. To the best of our knowledge, no prior MetaBBO works share the research objective with our work: meta learning flexible and generalizable constraint handling policy for constrained BBO.

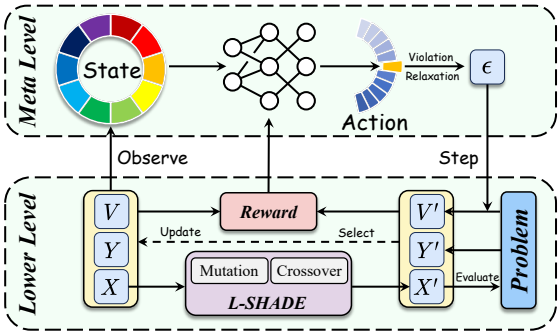

Fig. 1: The general bi-level workflow of MeCO.

3 Methodology

3.1 Overview

The overall bi-level workflow of MeCO is illustrated in Fig. 1. We formulate the optimization process of the low-level optimizer (L-SHADE [27] with ϵ-relaxation Comparison rule [26]) as a MDP:

$$\mathcal{M} := \langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T}, \pi_\theta \rangle \tag{4}$$

First, we observe current population P_t with N individuals to calculate current state $\boldsymbol{s}_t \in \mathcal{S}$. Then we feed the $\boldsymbol{s}_t$ into the current RL agent π_θ. The RL agent then outputs a reasonable action $a_t \in \mathcal{A}$, which determines the value of ϵ-relaxation for current optimization step. Next, the L-SHADE optimizer Λ uses the suggested ϵ-relaxation value to evolve P_t toward the next generation P_{t+1}. After that, we calculate the reward $r_t = \mathcal{R}(\boldsymbol{s}_t, a_t)$ to guide the learning of the RL agent's model parameters. To ensure the generalization, we facilitate the meta learning of π_θ by training MeCO on a problem distribution $\mathcal{D}$, with the following learning objective:

$$\theta^* := \arg\max \frac{1}{F} \sum_{k=1}^{F} \sum_{t=1}^{T} \gamma^{t-1} \mathcal{R}(\boldsymbol{S}_t, a_t | D_k), \tag{5}$$

which is the expected discounted accumulated performance gain across the problem distribution $\mathcal{D}$. We next elaborate each part of MeCO in detail.

3.2 MDP Designs

3.2.1 State. In MeCO, we profile the per-step optimization state of the low-level constrained optimization process as a 10-dimensional vector $\boldsymbol{s} \in \mathbb{R}^{10}$. We present the computation rules of these ten features as below:

- s_1: std $\left(\frac{x_i - x^L}{x^U - x^L}\right)$: The standard deviation of the individuals in the population, normalized by the search space boundaries x^L and x^U.
- s_2: std $\left(\frac{f(x_i) - f_{\text{best}}}{f_{\text{worst}} - f_{\text{best}}}\right)$: The standard deviation of the individual objective values at generation t, normalized by the historical best f_{best} and worst f_{worst} values.
- s_3: mean $\left(\frac{x_i - x^L}{x^U - x^L}\right)$: The mean of the individuals in the population, normalized by the search space boundaries.
- s_4: mean $\left(\frac{f(x_i) - f_{\text{best}}}{f_{\text{worst}} - f_{\text{best}}}\right)$: The mean of the individual normalized objective values at generation t.
- s_5: $\frac{f_{\text{best}}^{(t)}}{f_{\text{best}}^{(0)}}$: The optimization progress of the objective function relative to the initial generation. $f_{\text{best}}^{(t)}$ represents the minimum objective value found in the population at generation t.
- s_6: $\frac{\text{FEs}}{\text{FEs}_{\max}}$: The ratio of the consumed function evaluations (FEs) to the maximum available budget ($\text{FEs}_{\max}$), reflecting the current stage of the optimization process.
- s_7: $\frac{N_{\text{feas}}}{N_{\text{pop}}}$: The proportion of feasible solutions in the current population, where N_{feas} is the number of feasible individuals and N_{pop} is the total population size.
- s_8: $\frac{\bar{\nu}^{(t)}}{\bar{\nu}^{(0)}}$: The reduction progress of constraint violations, where $\bar{\nu}^{(t)}$ denotes bthe average of the top 5 smallest violation values in the population at optimization step t.
- s_9: ϵ_{t-1}: The numerical value of the ϵ-relaxation level applied in the previous step, see Sec. 3.2.2 for more elaboration on ϵ_t.
- s_{10}: $\frac{2}{N(N-1)} \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} \mathbb{I}\left(\frac{f(\mathbf{x}_i) - f(\mathbf{x}_j)}{\nu(\mathbf{x}_i) - \nu(\mathbf{x}_j)} > 0\right)$, which is the trade-off relationship between constraint violation and objective function values within the current population.

These ten features are categorized into two groups. The first group ($s_1 \sim s_6$) comprises features widely adopted in MetaBBO methods [50, 58], where we incorporate Fitness Landscape Analysis (FLA) [59] and Exploratory Landscape Analysis (ELA) [60] to profile the optimization state from decision space and objective space. Despite these classic ELA features, to make MeCO compatible with constrained BBO environment, we additionally propose four specific features ($s_7 \sim s_{10}$) to enhance the state representation with constraint-related information. Note that these four features play a key role in MeCO's learning system, which we will provide further analysis in experiments.

3.2.2 Action. In MeCO, the meta-level policy dynamically control the ϵ-relaxation value of the low-level L-SHADE optimization process. Specifically, we propose an easy-to-use ϵ-relaxation action space $\mathcal{A}$ to facilitate smooth control strength. We construct $\mathcal{A}$ as a discrete action space $\mathcal{A} : \{0, 0.1, 0.2, ..., 1\}$ with 11 optional actions. At a given optimization step t, the meta-level agent π_θ maps

the state s_t to one of the actions: $a_t = \pi_\theta(s_t)$, then the actual ϵ-relaxation value for current step is decoded by:

$$\epsilon_t = (\epsilon_{\text{base}})^{a_t} \cdot \delta^{1-a_t} \tag{6}$$

where the $\boldsymbol{\epsilon}_{\text{base}}$ is a $p+q$-dimensional vector that records the average violation values over all solutions in the L-SHADE's initialized population, for the $p+q$ constraints in the optimized problem. δ is a very small threshold value (1e-3 in this paper). If a smaller a_t is selected, i.e., $a_t = 0$, then the ϵ_t is set to δ, which prompts L-SHADE optimizer to focus more on constraint violation reduction. In contrast, if a larger a_t is selected, i.e., $a_t = 1$, the low-level optimizer focuses more on optimization in objective space. By flexibly adjusting the tradeoff between objective and constraint, the overall optimization process could further balance exploration and exploitation, chasing a better optimization performance.

3.2.3 Reward. Existing MetaBBO works mainly focus on unconstrained optimization problems, hence they design reward functions without the consideration of interplay between the objective and constraints. To bridge this gap, we propose a novel weighted reward in MeCO to synergize performance feedback from both the objective reducing progress and constraint satisfaction progress. To be specific, the reward function is computed as:

$$r_t = \frac{1}{2}(1-\omega^{(t)})\frac{f^{*,(t)} - f^{*,(t-1)}}{f^{*,(0)} - f^*} + \frac{1}{2}\frac{\bar{\nu}^{(t-1)} - \bar{\nu}^{(t)}}{\bar{\nu}^{(0)}}. \tag{7}$$

The first part of r_t follows the common practice in existing MetaBBO works [50] to reflect the optimal searching progress per step. Here, $f^{*,(t)}$ is the best objective value found so far, f^* is the global optimal of the given optimization problem. The second part is specifically designed by us to reflect the progress of constraint violation reduction, where $\bar{\nu}^{(t)}$ denotes the average of the smallest five violation values at t-step solution population. The weight factor $\omega^{(t)}$ is computed as $\frac{\bar{\nu}^{(t)}}{\bar{\nu}^{(0)}}$. This weight factor ensures that: when constraint violation is hard to reduce ($\omega^{(t)}$ is quite large), the meta-level policy will be guided to focus more on optimal objective searching, and vice versa. To summarize, our proposed weighted reward function strikes a delicate balance between feasible region exploration and objective optimization.

3.3 Training of Workflow

3.3.1 Training Algorithm. In this paper, we adopt Double Deep Q-Network (DDQN) [61] to learn the meta-level policy π_θ in MeCO. For each problem instance $\mathcal{D}_k$ in a training problem set $\mathcal{D}$, π_θ sequentially interacts with the optimization environment over an entire episode. At each step t, the current state $\boldsymbol{s}_t$ is observed, based on which π_θ estimates the action-value vector $\mathbf{Q}(\boldsymbol{s}_t, a_t; \theta)$. An action $a_t \in \mathcal{A}$ is then selected using a ξ-greedy policy, determining the ϵ-relaxation value at step t. Then the low-level L-SHADE optimizer optimizes the

Algorithm 1: Pseudocode for meta-training MeCO

Input: Training set $\mathcal{D}$, online policy θ, replay memory $M(n)$, optimizer Λ, epochs $MaxEpoch$, optimization steps T, learning rate η, update frequency C;
Output: Trained policy θ^*;
Initialize target policy $\theta' = \theta$;
for $epoch = 1$ **to** $MaxEpoch$ **do**
  for each problem instance in $\mathcal{D}$ **do**
    Initialize solution population P_0;
    for $t = 0$ **to** $T - 1$ **do**
      Get state $\boldsymbol{s}_t$ by P_t following Sec. 3.2.1;
      Select action a_t via ξ-greedy on $Q(\boldsymbol{s}_t, a_t; \theta_t)$;
      Evolve P_t to P_{t+1} using Λ with a_t;
      Compute reward r_t following Sec. 3.2.3;
      Save $(\boldsymbol{s}_t, a_t, r_t, \boldsymbol{s}_{t+1})$ into M;
      if $M.size \geq n$ **then**
        Sample n transitions from M, compute loss $\mathcal{L}(\theta)$ following Eq. 8;
        Update online policy by $\theta = \theta - \eta \nabla_\theta \mathcal{L}(\theta)$;
      end if
      Update target policy every C learning steps, $\theta' = \theta$;
    end for
  end for
end for
return $\theta^* = \theta$;

given problem instance for a step and the reward r_t is computed as elaborated above and the optimization state is updated to $\boldsymbol{s}_{t+1}$ accordingly. Given the transition $(\boldsymbol{s}_t, a_t, r_t, \boldsymbol{s}_{t+1})$, the policy π_θ is updated by Bellman backup loss:

$$\mathcal{L}(\theta) = MSE\big(r_t + \gamma Q(\boldsymbol{s}_{t+1}, \arg\max_{a'} Q(\boldsymbol{s}_{t+1}, a'; \theta); \theta'), Q(\boldsymbol{s}_t, a_t; \theta)\big), \tag{8}$$

where γ is the discount factor, θ and θ' are the online Q network and target Q network in DDQN. θ' is updated periodically by copying parameters from θ. The full training procedure is summarized in Algorithm 1.

3.3.2 Network Architecture. The online policy θ and target policy θ' are parameterized as two identical two-layer Multi-Layer Perceptron (MLP), featuring a hidden layer with a SELU activation $\phi(\cdot)$, followed by an output layer with a sigmoid activation $\sigma(\cdot)$. To attain Q values, the mathematical formulation of this architecture is defined as follows:

$$Q_t = \sigma\left(\mathbf{W}_2 \cdot \phi(\mathbf{W}_1 \boldsymbol{s}_t + \mathbf{b}_1) + \mathbf{b}_2\right) \tag{9}$$

where $\boldsymbol{s}_t$ denotes the optimization state in Sec. 3.2.1. The vector $Q_t \in \mathbb{R}^{11}$ represents the estimated Q values for all optional actions. To summarize, the learnable parameters include weight $\mathbf{W}_1 \in \mathbb{R}^{64\times 10}$, $\mathbf{W}_2 \in \mathbb{R}^{11\times 64}$ and bias $\mathbf{b}_1 \in \mathbb{R}^{64}$, $\mathbf{b}_2 \in \mathbb{R}^{11}$, for both θ and θ'.

4 Experimental Results and Analyses

In this section, we mainly discuss two major research questions: **RQ1:** Considering representative baselines, can MeCO achieve superior optimization & generalization performance? **RQ2:** What has MeCO learned and what impacts its learning? Below, we first introduce the experimental settings and then address these two RQs respectively.

4.1 Experiment Setup

4.1.1 Baselines. To ensure comprehensive comparison, we include 7 diverse baselines tailored for constrained BBO: 1) **LS**: original L-SHADE [27] optimizer with penalty function ($\lambda = 1$) constraint handling; 2) **LS-BOC** [15]: L-SHADE with an adaptive penalty function; 3) **LS-ϵ**: L-SHADE with a fixed ϵ-relaxation value, set as 0.1 times the maximal violation value in initialized population; 4) **MDE-CGO** [62]: a newly proposed state-of-the-art optimizer that adds priority in selection operator by the violation values; 5) **C^{2}oDE** [63]: a bi-stage optimizer that first uses violation-based selection priority then uses ϵ-relaxation policy; 6) **COBRA** [20]: the most representative surrogate-based algorithm for solving constrained BBO; 7) **NSGA-III** [64]: a multi-objective optimizer that transforms objective-constraints space as multi-objective space. These 7 baselines' settings strictly follow their original papers. We additionally add a **MeCO-u** baseline, which is the initialized MeCO without training. All experiments use Intel 6254 CPU with 64GB of RAM.

4.1.2 MeCO's Settings. For MeCO, the population size in the low-level L-SHADE optimizer is 50, optimization steps T is decided by the allowed function evaluations in different testing problems, which will be given later. In MeCO's training, the capacity of the replay memory M is 64, the sampling size $n = 8$, training epochs $MaxEpoch = 50$, the training optimizer is AdamW [65] with linear learning rate decay (5×10^{-3} to 1×10^{-4}), ξ-greedy policy with $\xi = 0.1$, discount factor $\gamma = 1$ to make policy evenly focus on short or long term effects, and the update frequency of the target Q network $C = 10$. We prepare two training problem sets $\mathcal{D}$ for MeCO: two versions of the CEC2017 Constrained Benchmark [66] (the problem dimension $D = 10$ and $D = 50$). This benchmark comprises 28 diverse and challenging constrained BBO problems hence serves as an ideal testbed for the experiments. The allowed maximal function evaluations on the benchmark problems are set as $50 \times D$.

Table 1: Leave-one-out generalization of compared baselines.

F	LS-ϵ	LS [27]	LS-BOC [15]	MDE-CGO [62]	C^{2}oDE [63]	COBRA [20]	NSGA-III [64]	MeCO-u	MeCO
F_1	2.74e+05 ±5.29e+04	1.27e+05 ±4.15e+04	1.09e+05 ±5.37e+04	2.61e+05 ±1.76e+04	1.64e+05 ±3.96e+04	3.41e+05 ±5.53e+04	2.46e+05 ±4.30e+04	2.40e+05 ±3.16e+04	**1.01e+05 ±4.54e+04**
F_2	2.68e+05 ±6.96e+04	1.26e+05 ±5.00e+04	1.19e+05 ±4.05e+04	2.82e+05 ±5.49e+04	1.96e+05 ±6.05e+04	3.89e+05 ±8.49e+04	1.08e+05 ±4.85e+04	2.10e+05 ±3.06e+04	**9.01e+04 ±2.61e+04**
F_3	1.12e+05 ±2.49e+04	2.69e+05 ±9.45e+04	2.17e+05 ±6.78e+04	1.13e+05 ±2.39e+04	**6.73e+04 ±1.39e+04**	1.46e+05 ±1.32e+04	1.54e+05 ±3.44e+04	2.12e+05 ±6.23e+04	1.86e+05 ±4.69e+04
F_4	1.72e+03 ±1.06e+02	1.14e+03 ±1.99e+02	1.42e+03 ±2.98e+02	1.04e+03 ±5.46e+01	1.75e+03 ±1.79e+02	2.13e+03 ±1.25e+02	1.40e+03 ±1.23e+02	1.76e+03 ±1.93e+02	**9.52e+02 ±1.85e+02**
F_5	4.65e+06 ±1.19e+06	1.67e+06 ±1.80e+06	1.41e+06 ±1.07e+06	1.92e+06 ±2.85e+05	2.93e+06 ±7.99e+05	1.19e+07 ±1.24e+06	5.26e+06 ±8.77e+05	5.67e+06 ±2.36e+06	**7.60e+05 ±5.72e+05**
F_6	8.16e+03 ±8.17e+02	8.94e+03 ±1.29e+03	9.38e+03 ±1.74e+03	3.45e+03 ±7.02e+01	8.08e+03 ±8.11e+02	8.60e+03 ±6.00e+02	4.06e+03 ±2.64e+02	7.11e+03 ±8.53e+02	**2.38e+03 ±1.14e+03**
F_7	6.37e+03 ±4.32e+02	4.93e+03 ±6.40e+02	5.46e+03 ±4.57e+02	6.70e+03 ±2.56e+02	4.60e+03 ±2.89e+02	5.55e+03 ±3.08e+02	**4.41e+03 ±5.07e+02**	5.49e+03 ±4.90e+02	4.66e+03 ±4.77e+02
F_8	3.11e+05 ±6.44e+04	1.73e+05 ±5.11e+04	1.43e+05 ±3.72e+04	2.48e+05 ±3.56e+04	1.53e+05 ±3.46e+04	2.76e+05 ±5.55e+04	1.32e+05 ±3.48e+04	3.46e+05 ±3.80e+04	**1.18e+05 ±3.01e+04**
F_9	9.92e+04 ±4.37e+04	5.23e+04 ±4.24e+04	3.65e+04 ±2.37e+04	4.50e+04 ±1.17e+04	1.51e+05 ±9.47e+04	6.85e+04 ±1.59e+04	6.41e+04 ±1.97e+04	2.97e+04 ±1.50e+04	**1.71e+04 ±6.10e+03**
F_{10}	7.54e+05 ±6.31e+04	**3.44e+05 ±1.17e+05**	4.03e+05 ±7.49e+04	5.86e+05 ±2.86e+04	5.00e+05 ±6.24e+04	8.56e+05 ±7.64e+04	6.06e+05 ±1.03e+05	4.71e+05 ±8.03e+04	3.67e+05 ±1.47e+05
F_{11}	3.32e+05 ±2.83e+04	**7.72e+04 ±3.84e+04**	1.67e+05 ±7.62e+04	3.24e+05 ±3.39e+04	3.02e+05 ±2.91e+04	2.62e+05 ±1.76e+04	2.16e+05 ±3.01e+04	1.95e+05 ±1.92e+04	9.09e+04 ±1.68e+04
F_{12}	2.00e+05 ±1.11e+04	7.42e+04 ±3.98e+04	7.21e+04 ±2.88e+04	1.12e+05 ±1.13e+04	1.12e+05 ±2.28e+04	2.85e+05 ±2.03e+04	1.72e+05 ±2.38e+04	1.19e+05 ±3.41e+04	**6.38e+04 ±3.09e+04**
F_{13}	4.62e+10 ±8.88e+09	1.70e+10 ±1.21e+10	1.46e+10 ±1.22e+10	2.06e+10 ±6.02e+09	2.28e+10 ±6.10e+09	1.07e+11 ±1.71e+10	5.46e+10 ±1.14e+10	1.92e+10 ±7.43e+09	**9.83e+09 ±8.15e+09**
F_{14}	1.89e+05 ±1.44e+04	9.92e+04 ±4.93e+04	9.30e+04 ±3.71e+04	3.97e+05 ±2.91e+04	1.39e+05 ±2.23e+04	2.81e+05 ±2.02e+04	2.33e+05 ±1.78e+04	1.44e+05 ±5.47e+04	**7.97e+04 ±4.28e+04**
F_{15}	9.16e+04 ±8.00e+03	4.60e+04 ±1.94e+04	3.80e+04 ±1.88e+04	9.15e+04 ±9.79e+03	5.97e+04 ±9.52e+03	1.38e+05 ±1.15e+04	1.21e+05 ±1.50e+04	2.16e+05 ±1.76e+04	**3.38e+04 ±2.24e+04**
F_{16}	8.94e+04 ±5.75e+03	2.95e+04 ±1.54e+04	3.75e+04 ±2.84e+04	5.12e+04 ±4.46e+03	6.51e+04 ±4.24e+03	1.37e+05 ±1.42e+04	9.98e+04 ±9.69e+03	7.95e+04 ±8.81e+03	**2.69e+04 ±1.78e+04**
F_{17}	9.93e+04 ±1.35e+04	5.14e+04 ±2.61e+04	4.06e+04 ±1.72e+04	5.29e+04 ±2.06e+03	6.34e+04 ±1.32e+04	1.35e+05 ±1.28e+04	9.22e+04 ±1.49e+04	4.98e+04 ±1.35e+04	**3.93e+04 ±1.81e+04**
F_{18}	4.86e+10 ±1.10e+10	1.01e+10 ±1.08e+10	1.08e+10 ±9.42e+09	1.34e+10 ±3.18e+09	3.33e+10 ±1.07e+10	1.17e+11 ±1.09e+10	4.64e+10 ±7.05e+09	6.23e+10 ±6.78e+9	**9.16e+09 ±4.18e+09**
F_{19}	7.25e+04 ±4.18e+01	7.22e+04 ±1.01e+02	7.22e+04 ±8.85e+01	7.22e+04 ±9.15e+01	7.23e+04 ±6.39e+01	7.24e+04 ±6.47e+01	7.21e+04 ±1.33e+02	7.24e+04 ±7.05e+01	**7.21e+04 ±7.53e+01**
F_{20}	1.94e+01 ±8.28e-01	1.84e+01 ±5.80e-01	1.85e+01 ±1.16e+00	1.75e+01 ±5.32e-01	1.75e+01 ±6.52e-01	1.80e+01 ±9.55e-01	1.77e+01 ±5.93e-01	1.89e+01 ±7.28e-01	**1.74e+01 ±1.18e+00**
F_{21}	1.92e+05 ±1.90e+04	7.42e+04 ±3.98e+04	7.21e+04 ±2.88e+04	1.09e+05 ±1.46e+04	1.18e+05 ±2.09e+04	2.84e+05 ±1.87e+04	1.71e+05 ±2.58e+04	9.22e+04 ±3.12e+04	**6.90e+04 ±3.40e+04**
F_{22}	4.53e+10 ±1.21e+10	8.47e+09 ±8.41e+09	1.60e+10 ±1.35e+10	2.13e+10 ±5.77e+09	2.52e+10 ±9.87e+09	1.04e+11 ±1.51e+10	4.44e+10 ±1.12e+10	1.94e+10 ±9.34e+10	**4.82e+09 ±1.87e+09**
F_{23}	2.00e+05 ±2.01e+04	7.45e+04 ±4.70e+04	1.03e+05 ±4.51e+04	4.26e+05 ±2.65e+04	1.32e+05 ±2.22e+04	2.74e+05 ±2.72e+04	2.34e+05 ±3.30e+04	9.38e+04 ±3.59e+04	**6.87e+04 ±3.84e+04**
F_{24}	8.90e+04 ±9.32e+03	4.26e+04 ±2.98e+04	3.86e+04 ±2.46e+04	1.07e+05 ±1.43e+04	6.05e+04 ±1.00e+04	1.34e+05 ±4.53e+03	1.10e+05 ±1.71e+04	4.15e+04 ±2.21e+04	**2.80e+04 ±2.41e+04**
F_{25}	9.09e+04 ±1.22e+04	5.34e+04 ±1.80e+04	5.27e+04 ±3.48e+04	5.96e+04 ±7.37e+03	5.93e+04 ±1.13e+04	1.42e+05 ±9.61e+03	4.04e+04 ±1.32e+04	2.18e+05 ±2.04e+04	**3.69e+04 ±1.72e+04**
F_{26}	9.35e+04 ±1.14e+04	5.14e+04 ±2.61e+04	4.06e+04 ±1.72e+04	5.58e+04 ±5.51e+03	6.21e+04 ±1.25e+04	1.37e+05 ±1.03e+04	8.85e+04 ±8.88e+03	5.02e+04 ±1.95e+04	**3.94e+04 ±1.78e+04**
F_{27}	4.50e+10 ±6.94e+09	**6.93e+09 ±6.73e+09**	1.37e+10 ±1.21e+10	1.49e+10 ±2.13e+09	3.15e+10 ±5.86e+09	1.16e+11 ±1.73e+10	4.48e+10 ±1.05e+10	7.16e+09 ±3.52e+09	7.01e+09 ±4.19e+09
F_{28}	7.25e+04 ±3.24e+01	7.24e+04 ±7.85e+01	7.23e+04 ±1.51e+02	7.24e+04 ±6.16e+01	7.25e+04 ±3.81e+01	7.25e+04 ±6.06e+01	7.25e+04 ±4.70e+01	7.25e+04 ±1.32e+02	**7.23e+04 ±1.24e+02**
Avg Rank	7.07	3.61	3.39	5.21	5.14	8.04	5.57	5.61	**1.36**

4.2 Optimization & Generalization Performance (RQ1)

4.2.1 Leave-one-out Generalization. We first validate MeCO's leave-one-out generalization performance on CEC2017 with $D = 50$. For each test case, we train MeCO on 27 problem instances and then test the trained model on the rest one. All baselines are tested on 28 cases, with 10 independent runs. The performance metric is computed as $\mathrm{f}(\boldsymbol{x}^*) + \nu(\boldsymbol{x}^*)$, where $\boldsymbol{x}^*$ is the best solution found by a baseline on a run, $\nu(\boldsymbol{x}^*)$ is the violation values in Eq. 2. We present

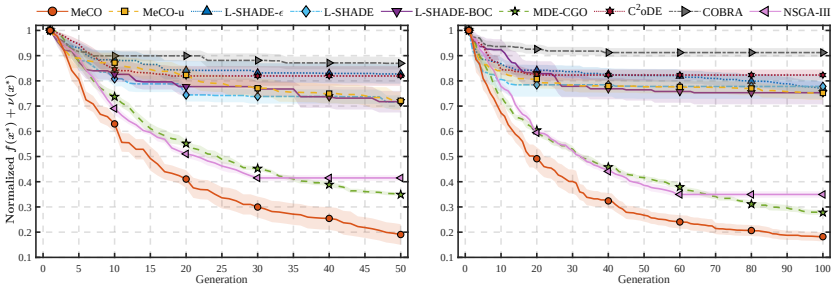

Fig. 2: The generalization optimization curves on 50D problems (left) and 100D problems (right), X-axis: optimization steps, Y-axis: normalized performance.

the average performance and error bars of baselines in Table 1, the best results are **bolded** and the second-best are <u>underlined</u>. The results show that:

(1) On almost all test cases, MeCO outperforms all baselines, achieving state-of-the-art performance and demonstrating consistent generalization ability on unseen problems. This further validates our research motivation: meta learning algorithm design could address limitations of those human-crafted ones, in both generalization and automation sides.
(2) Specifically, considering the comparison between MeCO-u and MeCO, a clear performance gap could be observed, which provides significant proof on the learning effectivess of MeCO.
(3) The compared human-crafted baselines such as LS and C^2oDE may surpass our MeCO on one or two specific test cases, however, they perform poorly on the majority, which indicates inherent biases and limited robustness of manually designed algorithms.

4.2.2 Dimension Generalization. A more challenging generalization scenario is about the problem dimension. To validate whether a MeCO trained on low-dimensional problems could be directly zero-shot toward high-dimensional problems, we train a MeCO on CEC2017 with $D = 10$, and then test the trained model on CEC 2017 with $D = 50$ and $D = 100$. We plot the average normalized optimization curves across the 28 testing problems in Fig. 2. Where the performance is normalized by the initial maximal violation values in a per-run fashion. This helps scale different problems' performance values to 0 to 1. From the results in Fig. 2, we can observe that:

(1) The strategy learned by the model on 10D problems remains effective on 50D and 100D problems. This suggests that high-dimensional and low-dimensional problem spaces share similar learnable features, enabling strategies learned in low-dimensional spaces to generalize to high-dimensional ones.
(2) The 7 baselines converge rapidly in the early stages of optimization. In contrast, MeCO's optimization curve remains unconverged while achieving

superior performance. This indicates that the inherent biases of manually designed algorithms make them prone to quickly falling into local optima, whereas MeCO, with its flexible and generalizable ϵ-relaxation policy, is less susceptible to local traps and can continuously locate better solutions.

4.2.3 Real-World Problem Generalization To further investigate the practical applicability of MeCO, we evaluate its generalization performance on a real-world constrained optimization task: UAV path planning for data collection from wireless sensor networks (WSNs) [62]. To better simulate complex physical environments, we artificially introduce obstacle constraints into the scenario and create 16 different tasks (we provide a example in the left of Fig. 3). We directly deploy the MeCO policy pre-trained on the CEC2017 with $D = 10$ to solve these task instances. For a fair comparison, the number of UAV hovering points is fixed at 6, and the allowed maximal function evaluations are strictly limited to 2500 for all baselines. In the right of Fig. 3, we present the average normalized performance(reversed by using one minus the normalized performance mentioned before) across 16 tasks and 10 independent runs. The results clearly indicate that MeCO outperforms the human-crafted baselines in handling this realistic constrained scenario. Benefiting from its dynamically adjusted ϵ-relaxation policyy, MeCO expertly balances constraint satisfaction and objective minimization. As a result, MeCO is capable of generating an optimal flight path that strictly avoids collision zones while guaranteeing complete information collection and successful arrival at the target destination.

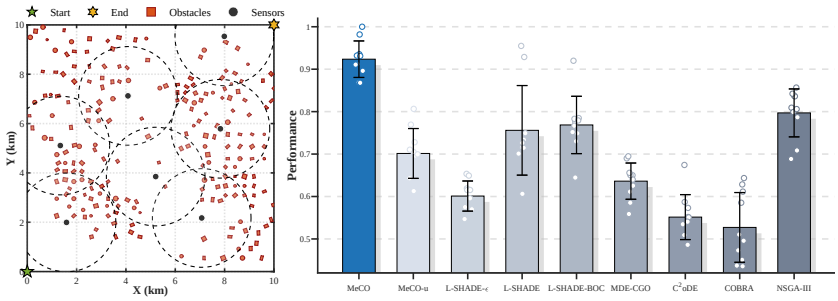

Fig. 3: The environment of the UAV path planning task and performance comparisons of the evaluated algorithms.

4.3 In-depth Analysis (RQ2)

4.3.1 Behavior Analysis. Fig. 4 illustrates the statistical distribution of ϵ-relaxation actions selected by MeCO for optimizing two mathematically different problem instances (F_{12} and F_{14}) in CEC2017 with $D = 10$. Through comparing the behavioral patterns on these two benchmark functions, we can

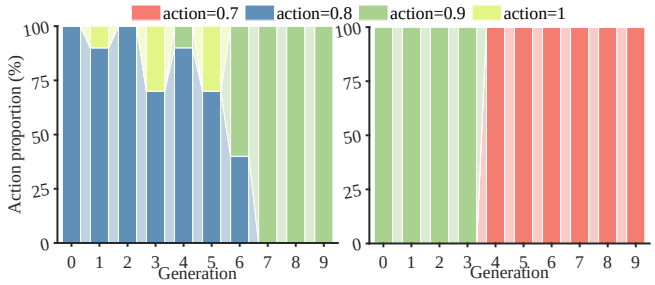

Fig. 4: The dynamics of ϵ-relaxation levels selected by the MeCO over 10 generations. Left: F_{12}, Right: F_{14}.

better understand the relationship between the meta-level policy and the low-level optimization landscapes. The mathematical expressions of the F_{12} and F_{14} benchmark functions are presented below:

Problem	**Objective Function** $f(\mathbf{x})$	**Inequality Constraint** $g(\mathbf{x})$	**Equality Constraint** $h(\mathbf{x})$
F_{12}	$\sum_{i=1}^{D}\left(y_i^2 - 10\cos(2\pi y_i) + 10\right)$	$4 - \sum_{i=1}^{D}\lvert y_i\rvert \leq 0$	$\sum_{i=1}^{D} y_i^2 - 4 = 0$
F_{14}	$\max_{1\leq i\leq D}\lvert y_i\rvert$	$\sum_{i=1}^{D} y_i^2 - 100D \leq 0$	$\cos(f_{15}(\mathbf{x})) + \sin(f_{15}(\mathbf{x})) = 0$

where $\mathbf{y} = \mathbf{x} - \mathbf{o} \in [-100, 100]^D$ is the shifted vector.

For F_{12}, the primary constraint is defined by the equality ($\sum_{i=1}^{D} y_i^2 - 4 = 0$), which restricts the search space to the surface of a hypersphere. Although the inequality constraint ($4 - \sum_{i=1}^{D} |y_i| \leq 0$) renders the regions near the coordinate axes infeasible, the remaining feasible region forms a connected manifold in high-dimensional space. This connectivity allows the optimizer to traverse the feasible surface continuously from one solution to another without leaving the feasible region. Therefore, given the simpler constraint geometry of F_{12}, MeCO shows a preference for higher ϵ-relaxation levels, thereby prioritizing the optimization of the objective function over constraint satisfaction. In contrast, the constraint of F_{14} is governed by $\cos(f(x)) + \sin(f(x)) = 0$. This dependency creates a topology consisting of disjoint, nested hypercube shells. The feasible region is not continuous but discrete; valid solutions exist only at specific "quantized" objective values, separated by vast infeasible gaps. Consequently, when facing F_{14}, which presents more challenging constraints, our RL agent tends to select lower ϵ-relaxation levels, concentrating more on finding feasible solutions.

4.3.2 Importance of State Representation. Recall that to make MeCO more compatible with constrained BBO, we have additionally proposed several constraint-related landscape features to enhance the state representation and learning effectiveness of our framework. To demonstrate the effectiveness of the

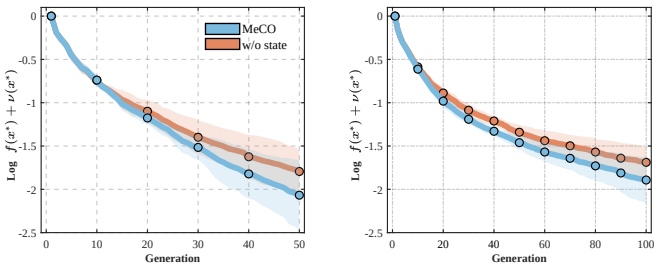

Fig. 5: Ablation on constraint-related state features, Left: 50D, Right: 100D.

constraint-related features, we ablate these features ($S_7 \sim S_{10}$) in MeCO's state feature design by masking them with zero values at the inference phase (testing), only keeping those not designed for profiling constraint violation status. We denote this ablation as MeCO *w/o state* and compare the resulting variant with the original design. For the concrete evaluation, we use the MeCO model pre-trained on CEC2017 with $D = 10$ and test it on high-dimensional CEC2017 sets ($D = 50$ and $D = 100$). The corresponding optimization curves are shown in Fig. 5. It can be observed that the performance of MeCO degrades consistently in both dimension-generalization settings after removing these features. This result not only indicates that the constraint-related features in the state representation are necessary, as they facilitate the learning of more effective policies for locating high-quality feasible solutions, but also reveals that the designed state features may help MeCO in its generalization toward more complex problems.

5 Conclusion

In this paper, we propose MeCO, a novel MetaBBO framework to meta-learn flexible and generalizable constraint handling policy for solving constrained BBO problems. For this purpose, we adapt the bi-level MetaBBO framework into the solving process of constrained BBO problems. In specific, MeCO maintains a learnable RL-based policy at the meta level capable of controlling the ϵ-relaxation level at the low-level L-SHADE optimizer. Through carefully designed optimization state features and reward feedback function, the meta-level policy and low-level optimization environment are bridged together. Once MeCO is meta-trained on a pre-defined problem distribution (set), the learned ϵ-relaxation policy could be directly used on unseen problems with desired optimization robustness. We validate MeCO's generalized optimization performance on both synthetic and realistic benchmarks, where it shows superior results to representative human-crafted baselines. Our work represents an early exploration on combining learning-based techniques into constrained BBO algorithms for further performance enhancement. As a pioneering work, MeCO has certain room for improvement. Specifically, optimizing the reward design and developing a more fine-grained state representation are our immediate future work.

References

1. D. R. Jones, "Large-scale multi-disciplinary mass optimization in the auto industry," in *MOPTA 2008 Conference (20 August 2008)*, vol. 64, 2008.
2. F. Boukouvala, M. F. Hasan, and C. A. Floudas, "Global optimization of general constrained grey-box models: new method and its application to constrained pdes for pressure swing adsorption," *Journal of Global Optimization*, vol. 67, no. 1, pp. 3–42, 2017.
3. C. A. C. Coello, "Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: a survey of the state of the art," *Computer methods in applied mechanics and engineering*, vol. 191, no. 11-12, pp. 1245–1287, 2002.
4. E. Mezura-Montes and C. A. C. Coello, "Constraint-handling in nature-inspired numerical optimization: past, present and future," *Swarm and Evolutionary Computation*, vol. 1, no. 4, pp. 173–194, 2011.
5. S. Shao, Y. Tian, and Y. Zhang, "Deep reinforcement learning assisted surrogate model management for expensive constrained multi-objective optimization," *Swarm and Evolutionary Computation*, vol. 92, p. 101817, 2025.
6. J. Liu, J. Chen, Z. Yang, Y. Liu, H. Qiu, and L. Gao, "Two-layer surrogate-assisted collaborative framework for expensive constrained optimization problems involving mixed integer variables," *Information Sciences*, vol. 690, p. 121522, 2025.
7. S. Alarie, C. Audet, M. Diago, S. L. Digabel, and X. Lebeuf, "Inter-ds: a cost saving algorithm for expensive constrained multi-fidelity blackbox optimization," *Computational Optimization and Applications*, vol. 90, no. 3, pp. 607–629, 2025.
8. E. Mezura-Montes and C. A. C. Coello, "Constraint-handling in nature-inspired numerical optimization: past, present and future," *Swarm and Evolutionary Computation*, vol. 1, no. 4, pp. 173–194, 2011.
9. Z.-H. Zhan, L. Shi, K. C. Tan, and J. Zhang, "A survey on evolutionary computation for complex continuous optimization," *Artificial Intelligence Review*, vol. 55, no. 1, pp. 59–110, 2022.
10. R. G. Regis, "A survey of surrogate approaches for expensive constrained black-box optimization," in *World Congress on Global Optimization*, pp. 37–47, Springer, 2019.
11. J. H. Holland, *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. MIT press, 1992.
12. J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95-international conference on neural networks*, vol. 4, pp. 1942–1948, ieee, 1995.
13. R. Storn and K. Price, "Differential evolution–a simple and efficient heuristic for global optimization over continuous spaces," *Journal of global optimization*, vol. 11, no. 4, pp. 341–359, 1997.
14. B. Tessema and G. G. Yen, "A self adaptive penalty function based algorithm for constrained optimization," in *2006 IEEE international conference on evolutionary computation*, pp. 246–253, IEEE, 2006.
15. T. Kawachi, J.-i. Kushida, A. Hara, and T. Takahama, "L-shade with an adaptive penalty method of balancing the objective value and the constraint violation," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pp. 5–6, 2019.
16. B. Tessema and G. G. Yen, "An adaptive penalty formulation for constrained evolutionary optimization," *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans*, vol. 39, no. 3, pp. 565–578, 2009.

17. T. Ray, H. K. Singh, A. Isaacs, and W. Smith, "Infeasibility driven evolutionary algorithm for constrained optimization," in *Constraint-handling in evolutionary optimization*, pp. 145–165, Springer, 2009.
18. Y. Wang, H. Liu, Z. Cai, and Y. Zhou, "An orthogonal design based constrained evolutionary optimization algorithm," *Engineering Optimization*, vol. 39, no. 6, pp. 715–736, 2007.
19. F. Samanipour and J. Jelovica, "Adaptive repair method for constraint handling in multi-objective genetic algorithm based on relationship between constraints and variables," *Applied Soft Computing*, vol. 90, p. 106143, 2020.
20. R. G. Regis, "Constrained optimization by radial basis function interpolation for high-dimensional expensive black-box problems with infeasible initial points," *Engineering Optimization*, vol. 46, no. 2, pp. 218–243, 2014.
21. R. G. Regis, "Surrogate-assisted particle swarm with local search for expensive constrained optimization," in *International Conference on Bioinspired Methods and Their Applications*, pp. 246–257, Springer, 2018.
22. Y. Li, J. Shen, Z. Cai, Y. Wu, and S. Wang, "A kriging-assisted multi-objective constrained global optimization method for expensive black-box functions," *Mathematics*, vol. 9, no. 2, p. 149, 2021.
23. T. Takahama and S. Sakai, "Constrained optimization by the ε constrained differential evolution with an archive and gradient-based mutation," in *IEEE congress on evolutionary computation*, pp. 1–9, IEEE, 2010.
24. Z. Fan, W. Li, X. Cai, H. Huang, Y. Fang, Y. You, J. Mo, C. Wei, and E. Goodman, "An improved epsilon constraint-handling method in moea/d for cmops with large infeasible regions," *arXiv preprint arXiv:1707.08767*, 2017.
25. Z. Yang, X. Cai, and Z. Fan, "Epsilon constrained method for constrained multi-objective optimization problems: some preliminary results," in *Proceedings of the companion publication of the 2014 annual conference on genetic and evolutionary computation*, pp. 1181–1186, 2014.
26. T. Takahama and S. Sakai, "Constrained optimization by the constrained differential evolution with gradient-based mutation and feasible elites," in *Proc. IEEE Congress on Evolutionary Computation*, vol. 1, 2006.
27. R. Tanabe and A. S. Fukunaga, "Improving the search performance of shade using linear population size reduction," in *2014 IEEE congress on evolutionary computation (CEC)*, pp. 1658–1665, IEEE, 2014.
28. R. S. Sutton, A. G. Barto, *et al.*, *Reinforcement learning: An introduction*, vol. 1. MIT press Cambridge, 1998.
29. G. Wu, R. Mallipeddi, and P. N. Suganthan, "Problem definitions and evaluation criteria for the cec 2017 competition on constrained real-parameter optimization," *National University of Defense Technology, Changsha, Hunan, PR China and Kyungpook National University, Daegu, South Korea and Nanyang Technological University, Singapore, Technical Report*, vol. 9, p. 2017, 2017.
30. J. J. Liang, T. P. Runarsson, E. Mezura-Montes, M. Clerc, P. N. Suganthan, C. C. Coello, and K. Deb, "Problem definitions and evaluation criteria for the cec 2006 special session on constrained real-parameter optimization," *Journal of Applied Mechanics*, vol. 41, no. 8, pp. 8–31, 2006.
31. C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International conference on machine learning*, pp. 1126–1135, PMLR, 2017.

32. Z. Ma, H. Guo, Y.-J. Gong, J. Zhang, and K. C. Tan, "Toward automated algorithm design: A survey and practical guide to meta-black-box-optimization," *IEEE Transactions on Evolutionary Computation*, 2025.
33. J. Chen, Z. Ma, H. Guo, Y. Ma, J. Zhang, and Y.-J. Gong, "Symbol: Generating flexible black-box optimizers through symbolic equation learning," *arXiv preprint arXiv:2402.02355*, 2024.
34. Z. Ma, H. Guo, J. Chen, Z. Li, G. Peng, Y.-J. Gong, Y. Ma, and Z. Cao, "Metabox: A benchmark platform for meta-black-box optimization with reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 10775–10795, 2023.
35. X. Li, K. Wu, X. Zhang, and H. Wang, "B2opt: Learning to optimize black-box optimization with little budget," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, pp. 18502–18510, 2025.
36. X. Li, K. Wu, X. Zhang, H. Wang, J. Liu, *et al.*, "Pretrained optimization model for zero-shot black box optimization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 14283–14324, 2024.
37. H. S. Gomes, B. Léger, and C. Gagné, "Meta learning black-box population-based optimizers," *arXiv preprint arXiv:2103.03526*, 2021.
38. R. Lange, T. Schaul, Y. Chen, T. Zahavy, V. Dalibard, C. Lu, S. Singh, and S. Flennerhag, "Discovering evolution strategies via meta-black-box optimization," in *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, pp. 29–30, 2023.
39. Z. Ma, Y.-J. Gong, H. Guo, J. Chen, Y. Ma, Z. Cao, and J. Zhang, "Llamoco: Instruction tuning of large language models for optimization code generation," *IEEE Transactions on Evolutionary Computation*, 2026.
40. N. van Stein and T. Bäck, "Llamea: A large language model evolutionary algorithm for automatically generating metaheuristics," *IEEE Transactions on Evolutionary Computation*, 2024.
41. F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang, "Evolution of heuristics: Towards efficient automatic algorithm design using large language model," *arXiv preprint arXiv:2401.02051*, 2024.
42. P. Li, J. Hao, H. Tang, X. Fu, Y. Zhen, and K. Tang, "Bridging evolutionary algorithms and reinforcement learning: A comprehensive survey on hybrid algorithms," *IEEE Transactions on evolutionary computation*, 2024.
43. J. Pei, Y. Mei, J. Liu, M. Zhang, and X. Yao, "Adaptive operator selection for meta-heuristics: A survey," *IEEE Transactions on Artificial Intelligence*, 2025.
44. P. Kerschke, H. H. Hoos, F. Neumann, and H. Trautmann, "Automated algorithm selection: Survey and perspectives," *Evolutionary computation*, vol. 27, no. 1, pp. 3–45, 2019.
45. G. Karafotias, M. Hoogendoorn, and Á. E. Eiben, "Parameter control in evolutionary algorithms: Trends and challenges," *IEEE Transactions on Evolutionary Computation*, vol. 19, no. 2, pp. 167–187, 2014.
46. H. Guo, Y. Ma, Z. Ma, J. Chen, X. Zhang, Z. Cao, J. Zhang, and Y.-J. Gong, "Deep reinforcement learning for dynamic algorithm selection: A proof-of-principle study on differential evolution," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 7, pp. 4247–4259, 2024.
47. Q. Guo, H. Wang, and Y. Tian, "Automated algorithm selection for black-box optimization using light gradient boosting machine," *Swarm and Evolutionary Computation*, vol. 98, p. 102071, 2025.

48. X. Liu, T. Wang, Z. Zeng, Y. Tian, and J. Tong, "Three stage based reinforcement learning for combining multiple metaheuristic algorithms," *Swarm and Evolutionary Computation*, vol. 95, p. 101935, 2025.
49. N. Zhu, F. Zhao, and J. Cao, "A hyperheuristic and reinforcement learning guided meta-heuristic algorithm recommendation," in *2024 27th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, pp. 1061–1066, IEEE, 2024.
50. Z. Ma, J. Chen, H. Guo, Y. Ma, and Y.-J. Gong, "Auto-configuring exploration-exploitation tradeoff in evolutionary computation via deep reinforcement learning," in *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 1497–1505, 2024.
51. H. Guo, S. Ma, Z. Huang, Y. Hu, Z. Ma, X. Zhang, and Y.-J. Gong, "Reinforcement learning-based self-adaptive differential evolution through automated landscape feature learning," in *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 1117–1126, 2025.
52. H. Guo, Z. Ma, J. Chen, Y. Ma, Z. Cao, X. Zhang, and Y.-J. Gong, "Configx: Modular configuration for evolutionary algorithms via multitask reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, pp. 26982–26990, 2025.
53. X. Yang, R. Wang, K. Li, and L. Wang, "Reinforcement learning based automated design of differential evolution algorithm for black-box optimization," *arXiv preprint arXiv:2501.12881*, 2025.
54. Y. Lu, Q. Tang, S. Yu, and L. Cheng, "A multi-strategy self-adaptive differential evolution algorithm for assembly hybrid flowshop lot-streaming scheduling with component sharing," *Swarm and Evolutionary Computation*, vol. 92, p. 101783, 2025.
55. J. Pei, Y. Mei, J. Liu, and M. Zhang, "Libog: Lifelong learning for black-box optimizer generation," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25* (J. Kwok, ed.), pp. 8912–8920, International Joint Conferences on Artificial Intelligence Organization, 8 2025. Main Track.
56. Q. Zhao, T. Liu, B. Yan, Q. Duan, J. Yang, and Y. Shi, "Automated metaheuristic algorithm design with autoregressive learning," *IEEE Transactions on Evolutionary Computation*, 2024.
57. X. Yang, R. Wang, K. Li, and H. Ishibuchi, "Meta-black-box optimization for evolutionary algorithms: Review and perspective," *Swarm and Evolutionary Computation*, vol. 93, p. 101838, 2025.
58. M. Sharma, A. Komninos, M. López-Ibáñez, and D. Kazakov, "Deep reinforcement learning based parameter control in differential evolution," in *Proceedings of the genetic and evolutionary computation conference*, pp. 709–717, 2019.
59. E. Pitzer and M. Affenzeller, "A comprehensive survey on fitness landscape analysis," *Recent advances in intelligent engineering systems*, pp. 161–191, 2012.
60. O. Mersmann, B. Bischl, H. Trautmann, M. Preuss, C. Weihs, and G. Rudolph, "Exploratory landscape analysis," in *Proceedings of the 13th annual conference on Genetic and evolutionary computation*, pp. 829–836, 2011.
61. H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, 2016.
62. Y. Bai, P.-F. Sun, T.-H. Wang, B. Sun, W.-J. Yu, J.-H. Zhong, G.-H. Song, S.-W. Jeon, S. T. W. Kwong, and J. Zhang, "Uav path planning for data collection

from wireless sensor network with matrix-based evolutionary computation," *IEEE Transactions on Intelligent Transportation Systems*, 2025.

63. B.-C. Wang, H.-X. Li, J.-P. Li, and Y. Wang, "Composite differential evolution for constrained evolutionary optimization," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 49, no. 7, pp. 1482–1495, 2018.
64. K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part i: solving problems with box constraints," *IEEE transactions on evolutionary computation*, vol. 18, no. 4, pp. 577–601, 2013.
65. I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*.
66. G. Wu, R. Mallipeddi, and P. Suganthan, "Problem definitions and evaluation criteria for the cec 2017 competition and special session on constrained single objective real-parameter optimization," *Nanyang Technol. Univ., Singapore, Tech. Rep*, pp. 1–18, 2016.